

# Final Çalışma Kitapçığı — Sıfırdan Anlatım

YAP 470 • Hiçbir ön bilgi varsaymaz. Her kavram aynı şablonla anlatılır: **Nedir?** → **Nasıl çalışır?** → **Projede ne işe yaradı?** • Kitapçık boyunca tek bir benzetme kullanılır: **MODEL = SINAVA HAZIRLANAN BİR ÖĞRENCİ**. Bunu bir kez kavrayınca her şey yerine oturur.

## BÖLÜM 0 — Büyük Resim: Bu Projede Aslında Ne Yapıyoruz?

Elimizde 7.442 kısa ses kaydı var. Her kayıta bir oyuncu, bir cümleyi 6 duygudan biriyle söylüyor (kızgın, tiksinti, korku, mutlu, nötr, üzgün). Amacımız: bilgisayara yeni bir kayıt verdiğimizde **"bu kişi hangi duyguyla konuşuyor?"** sorusuna doğru cevap veren bir program yapmak.

Ama bu programın kurallarını biz elle yazmıyoruz ("eğer ses yüksekse kızgındır" gibi). Onun yerine **makine öğrenmesi** kullanıyoruz: programa binlerce örnek gösteriyoruz ("bu kayıt kızgın, bu kayıt üzgün...") ve program kuralları **kendisi çıkarıyor**.

🎓 Ana benzetmemiz: Model, sınava hazırlanan bir öğrencidir.

- **Eğitim verisi** = ders çalışırken çözdüğü, cevabı verilmiş sorular.
- **Geçerleme (validation) verisi** = deneme sınavı — çalışma yöntemini buna göre ayarlar.
- **Test verisi** = gerçek sınav — hayatında bir kez girer, notu budur.

Bu kitapçıkta hemen her kavram bu benzetmeyle açıklanacak.

Projede bu "öğrenciden" iki tane yetiştirdik ve yarıştırdık:

- **Yöntem 1 (CNN):** sesi bir **resme** çevirip resim okuyan bir öğrenci.
- **Yöntem 2 (BiGRU):** sesi bir **zaman serisine** çevirip hikâye gibi baştan sona okuyan bir öğrenci.

Sonuç: BiGRU %63,8 ile kazandı, CNN %62,6 (SpecAugment ile %62,9). İnsanlar aynı işte %40,9 yapabiliyor.

## BÖLÜM 1 — Temel Kavramlar (her şeyin üzerine kurulduğu tuğlalar)

### 📌 MODEL

#### NEDİR?

Girdi alıp (bizde: ses) çıktı üreten (bizde: 6 duygudan biri), içi **ayarlanabilir sayılarla** (parametre/ağırlık) dolu bir matematik makinesi. "Öğrenmek" = bu sayıların doğru değerlere gelmesi.

Öğrencinin beyni. Başta bomboş (rastgele sayılar); çalışıkça bilgi (doğru sayılar) oturur.

#### PROJEDE

İki modelimiz var: CNN (~287 bin ayarlanabilir sayı) ve BiGRU (942.342 sayı). İkisi de sıfırdan, yani boş beyinle başladı — hazır model kullanmak yasaktı.

## 📌 EĞİTİM (TRAINING)

### NEDİR?

Modele örnekleri tekrar tekrar gösterip her seferinde ayar sayılarını "biraz daha doğru" yöne itme süreci.

### NASIL ÇALIŞIR?

1. Modele bir grup örnek ver (yığın/batch).
2. Model tahmin yapsın.
3. Tahmin ile doğru cevabın farkını ölç (loss).
4. Bu farkı azaltacak yönde tüm ayar sayılarını minicik güncelle.
5. Tüm veriyi bitir (1 epoch), başa dön, tekrarla.

Öğrenci soru çözer → cevabına bakar → nerede yanlış olduğunu görür → o konuyu biraz düzeltir → sonraki soruya geçer. Binlerce kez.

### PROJEDE

Her model 60 epoch'a kadar eğitildi ama erken durdurma genelde 15-35. epoch'ta kesti (aşağıda).

## 📌 EPOCH ve BATCH

### NEDİR?

**Epoch:** eğitim verisinin TAMAMININ modelden bir kez geçmesi. **Batch (yığın):** veriyi teker teker değil gruplar hâlinde işlemek (bizde 32-64'lük gruplar).

Epoch = soru bankasını baştan sona bir tur bitirmek. Batch = soruları 32'şerlik setler hâlinde çözüp her set sonunda kendini düzeltmek — tek tek çözüp her soruda düzeltmekten daha hızlı ve dengeli.

### PROJEDE

5.234 eğitim kaydı, 32'lik batch'lerle → epoch başına ~164 güncelleme adımı.

## 📌 LOSS (KAYIP / HATA FONKSİYONU)

### NEDİR?

Modelin ne kadar yanlış olduğunu ölçen TEK bir sayı. Eğitimin bütün amacı bu sayıyı küçültmek.

### NASIL ÇALIŞIR?

Bizde **cross-entropy**: model her sınıfa bir olasılık verir;  $loss = -\log(\text{doğru sınıfa verdiği olasılık})$ . Doğruya %100 derse loss 0; doğruya %1 derse loss kocaman. Yani "emin ve yanlış" olmak en pahalısı.

Sınav puanının tersi gibi: 0 = mükemmel, büyük sayı = kötü. Öğrenci hep "puan kaybımı azaltayım" diye çalışır.

### PROJEDE

Sınıf-ağırlıklı cross-entropy kullandık (Bölüm 5'te formülü ve gerçek sayılarla hesabı var).

## 📌 GRADYAN ve GRADYAN İNİŞİ

### NEDİR?

**Gradyan:** "her bir ayar sayısını hangi yöne oynatırsam loss azalır?" bilgisini veren pusula (matematikte: türev). **Gradyan inişi:** her adımda bu pusulanın gösterdiği yönde küçük bir adım atmak.

*Sisli bir dağda en alçak noktaya (en düşük hataya) inmeye çalışıyorsun. İleriye göremiyorsun ama ayağının altındaki eğimi hissediyorsun → hep yokuş aşağı yönde küçük adımlar atarsın.*

### NASIL ÇALIŞIR?

PyTorch, loss'tan geriye doğru zincir kuralıyla her parametrenin türevini otomatik hesaplar (**geri yayılım / backpropagation**). Sonra optimizer parametreleri günceller:  $\text{yeni\_ağırlık} = \text{eski\_ağırlık} - \text{öğrenme\_oranı} \times \text{gradyan}$ .

### PROJEDE

Optimizer olarak **AdamW** kullandık: her parametreye kendi geçmişine göre uyarlanmış adım boyu verir (sabit adımdan daha akıllı) + büyük ağırlıkları küçültme cezası (weight decay) ekler.

## 📌 ÖĞRENME ORANI (LEARNING RATE)

### NEDİR?

Her güncellemedeki adımın büyüklüğü. Bizde 0,001 gibi küçük sayılar.

*Dağdan inerken adım boyun: çok büyük adım → vadiyi atlayıp karşı yamaca çıkarsın (eğitim patlar); çok küçük adım → inmen yıllar alır. Doğru boy şart.*

### PROJEDE

Hiperparametre olarak arandı: 1e-4, 3e-4, 1e-3 denendi; kazananlar 1e-3 bandında.

## 📌 HİPERPARAMETRE

### NEDİR?

Modelin eğitimle ÖĞRENMEDİĞİ, bizim baştan seçtiğimiz ayarlar: katman sayısı, öğrenme oranı, batch boyutu, dropout oranı, **aralık sayısı ve genişliği**...

*Öğrencinin bilgisi = parametre (çalışarak edinir). Çalışma PROGRAMI = hiperparametre (günde kaç saat, hangi teknikle — bunu öğrenci değil, koç seçer). İyi program kötü programdan çok daha iyi öğrenci çıkarır.*

### PROJEDE

43 farklı "çalışma programı" (aday) denedik; hangisinin iyi olduğuna DENEME SINAVI (geçerleme) puanıyla karar verdik. RNN'i birinci yapan şey de bir hiperparametre bulgusuydu: 32 aralık × 200 ms.

## ✦ EĞİTİM / GEÇERLEME / TEST BÖLMESİ

### NEDİR?

Veriyi üçe ayırırız: **eğitim** (model bundan öğrenir), **geçerleme/validation** (ayarları buna göre seçeriz), **test** (en sonda, bir kez, gerçek başarıyı ölçer).

### NEDEN ÜÇ PARÇA?

Öğrenciyi çalıştığı sorularla sınarsan ezberi ölçersin, bilgiyi değil. Deneme sınavıyla yöntem seçersin; ama deneme sınavına göre 43 kez ayar yaptıysan artık deneme puanın da biraz "şişkin"dir — o yüzden hiç görülmemiş GERÇEK sınav (test) şarttır ve ona yalnız bir kez girilir.

### PROJEDE

5.234 eğitim / 1.148 geçerleme / 1.060 test. Test'e yöntem başına yalnızca nihai kazanan dokundu.

## ✦ KONUŞMACI-BAĞIMSIZ BÖLME (speaker-independent)

### NEDİR?

Bölmeyi kayıtlara göre değil **KİŞİLERE göre** yapmak: bir aktörün TÜM kayıtları ya eğitimde ya testte — asla ikisinde birden değil.

### NEDEN HAYATİ?

Aynı kişinin sesi hem eğitimde hem testte olursa model duyguyu değil **o kişinin ses rengini** ezberler ve testte "tanıdık sesi hatırlayarak" doğru cevaplar → skor sahte şişer (literatürde 20-30+ puan!).

*Gerçek sınava, çalışırken gördüğün soruların birebir aynılarının gelmesi gibi: notun yüksek çıkar ama hiçbir şey öğrenmemişsindir. İnternetteki %90+ CREMA-D sonuçları bu tuzaktan.*

### PROJEDE

91 aktör kimliğine göre böldük (kod: `ser/data/splits.py`). Sunumdaki en kritik savunma noktan bu.

## ✦ DATA LEAKAGE (VERİ SIZINTISI)

### NEDİR?

Test hakkındaki HERHANGİ bir bilginin, eğitim/karar sürecine gizlice karışması. Konuşmacı karışması bunun bir türü; başka türleri de var.

*Sınav sorularının önceden sızması. Kopya çeken öğrencinin notu gerçek bilgisini göstermez.*

### PROJEDEKİ 3 ÖNLEM

1. Konuşmacı-bağımsız bölme (yukarıda).
2. Normalizasyon (z-skor) SADECE eğitim verisinden öğrenildi — test istatistikleri hiçbir hesaba girmedir.
3. Veri artırma (SpecAugment) bölmeden SONRA ve yalnız eğitim yığınlarına uygulandı.

## 📌 OVERFITTING (AŞIRI ÖĞRENME)

### NEDİR?

Modelin, eğitim verisindeki genel örüntüler yerine o verinin **ezberini** yapması: eğitim puanı artmaya devam ederken YENİ veride puanın düşmesi.

*Soru bankasının CEVAP ANAHTARINI ezberleyen öğrenci: bankadaki soruda %100, gerçek sınavda çuval.*

### NASIL FARK EDİLİR?

Öğrenme eğrisinden: eğitim loss'u düşerken geçerleme loss'u önce düşer, sonra DÖNER ve yükselmeye başlar. Dönüş noktası = ezberin başladığı an. Bizim CNN grafiğinde bu ~12. epoch'ta net görülüyor (eğitim macro-F1 0,90'a çıkarken geçerleme 0,55'te kaldı!).

### PROJEDEKİ PANZEHİRLER

Early stopping (aşağıda) + dropout + weight decay + SpecAugment + küçük model tercihi. Hepsi ezbere karşı.

## 📌 EARLY STOPPING (ERKEN DURDURMA)

### NEDİR?

Deneme sınavı (geçerleme) puanı düzelmeyi bıraktığında eğitimi kesmek ve **en iyi anın modelini** almak.

### NASIL ÇALIŞIR? (bizim adımlarımız)

1. Her epoch bitince geçerleme macro-F1 ölç.
2. Şimdiye kadarki en iyiye: modelin o anki ayar sayılarının KOPYASINI kenara koy.
3. Üst üste 8 epoch (patience=8) anlamlı iyileşme yoksa (min\_delta=0,0001) → DUR.
4. Kenara koyduğun EN İYİ kopyayı geri yükle. Dönen model, eğitimin sonundaki değil, deneme sınavında en iyi olan modeldir.

*Koç, öğrencinin deneme netleri 8 hafta üst üste artmayınca "dur, ezbere kaçtın" der ve öğrenciyi en iyi hâline (o haftaki çalışma durumuna) geri döndürür.*

### PROJEDE

CNN kazananı 37 epoch koştı, en iyisi 29. epoch'tu → 29'un modeli alındı. RNN 9. epoch'ta zirveydi. Kod: `final/training.py`.

## ✦ Z-SKOR NORMALİZASYONU (Standardizasyon)

### NEDİR?

Her özneliği "(değer – ortalama) / standart sapma" yaparak ortalaması 0, yayılımı 1 olan ortak bir ölçeğe çekmek.

### NEDEN GEREK?

Özneliklerin biri 0-1 arasında, öbürü 0-8000 arasında olabilir (ör. ZCR vs centroid). Ölçeklenmezse büyük sayılı öznelik eğitime haksız hükmeder; ağırlar küçük-dengeli sayılarla çok daha iyi öğrenir.

*Farklı derslerin notlarını (biri 5'lik, biri 100'lük sistem) kıyaslamak için hepsini aynı cetvele çevirmek.*

### KRİTİK KURAL

Ortalama/std SADECE eğitim kümesinden hesaplanır; geçerleme ve test AYNİ sayılarla dönüştürülür. Testten istatistik almak = sızıntı. Kod: `final/dataset.py` içindeki Standardizer.

## ✦ SEED (TOHUM) ve TEKRARLANABİLİRLİK

### NEDİR?

Bilgisayarın "rastgele" sayıları aslında bir başlangıç değerinden (seed) üretilir. Seed'i sabitlersen (bizde 42) tüm rastgelelik — bölme, karıştırma, başlangıç ağırlıkları — her koşuda AYNİ olur.

### PROJEDE

Bu sayede notebook, kaydedilmiş modeli yükleyip testi yeniden koştuğunda kayıtlı sonuçlarla **birebir aynı** sayıları üretiyor — "tekrar üretilebilirlik" şartının kanıtı.

## BÖLÜM 2 — Ses Nasıl Sayıya Dönüşür? (iki yöntemin ortak girişi)

## ✦ ÖRNEKLEME (SAMPLING)

### NEDİR?

Ses havada sürekli bir titreşimdir; mikrofon+bilgisayar bunu saniyede binlerce kez ölçüp sayı listesine çevirir. Biz saniyede **16.000 ölçüm** (16 kHz) kullandık.

### NEDEN 16.000?

Matematik kuralı (Nyquist): saniyede N ölçümle en fazla N/2 frekansı temsil edebilirsin. 16k → 8.000 Hz'e kadar; konuşmadaki duygu bilgisinin neredeyse tamamı bu bandın içinde. Daha yükseği boşuna yük olurdu.

*Dönen bir tekerleğe saniyede kaç kez fotoğraf çekersen hızını doğru görürsün? Az çekersen teker geriye dönüyor sanırsın (film tekerleği illüzyonu) — yeterince sık ölçmek şart.*

## ✦ SPECTROGRAM ve STFT

### NEDİR?

Sesin **zaman × frekans haritası**: yatay eksen zaman, dikey eksen frekans (ince/kalın), renk o anda o frekansta ne kadar enerji olduğu.

### NASIL ÜRETİLİR? (STFT)

1. Ses listesini kısa pencerelere böl (bizde 1024 örnek = 64 ms).
2. Her pencerede Fourier dönüşümü uygula → "bu 64 ms'de hangi frekanslar vardı?"
3. Pencereyi 256 örnek (16 ms) kaydır, tekrarla.
4. Sütunları yan yana diz → harita çıkar.

*Bir şarkının nota kâğıdı gibi: soldan sağa zaman akar, yukarıda tiz notalar, aşağıda pes notalar; kalın yazılmış nota = güçlü ses.*

### PROJEDE

Yöntem 1'in girdisinin ham hâli. Fourier'in kendisi ezberlenecek değil; "sesi frekans bileşenlerine ayıran matematik" demen yeterli.

## ✦ MEL ÖLÇEĞİ ve LOG-MEL

### NEDİR?

İnsan kulağının frekans algısını taklit eden ölçek. Kulak 100→200 Hz farkını dev, 7000→7100 Hz farkını hiç duymaz. Mel ölçeği bu yüzden alçak frekanslara SIK, yükseklerle SEYREK 64 filtre yerleştirir. Sonra enerjinin logaritması alınır (kulak şiddeti de logaritmik duyar) → **log-mel spectrogram**.

*Cetveli kulağın algısına göre bükme: kulağın hassas olduğu bölgeye milimetrik çizgiler, duyarsız bölgeye santimetrik.*

### PROJEDE

Yöntem 1'in girdisi: **64 mel bandı × 128 zaman karesi**'lik bir "resim". Uzun kayıt ortadan kırpılır, kısa kayıt doldurulur (her resim aynı boyutta olsun diye). Kod: `final/features.py` → `extract_mel_image`.

## ✦ MFCC ve DELTA-MFCC

### NEDİR?

MFCC = log-mel spektrumunun üstüne bir sıkıştırma dönüşümü (DCT) daha uygulayıp ilk 13 katsayıyı almak. Bu 13 sayı, sesin **tınısını** (ağız-boğaz şeklinin sese verdiği renk) çok kompakt özetler — konuşma işlemenin 40 yıllık standart "parmak izi"dir. **Delta-MFCC** = bu katsayıların zamanla değişim hızı (konuşmanın hareketliliği).

*Spectrogram = yüzün fotoğrafı; MFCC = o yüzün 13 sayılık eşkâl tarifi (göz rengi, burun uzunluğu...). Az sayıyla çok bilgi.*

### PROJEDE

Yöntem 2'nin aralık başına 44 sayısının 39'u MFCC ailesinden: 13 ortalama + 13 std + 13 delta ortalaması.



## SON 5 İSTATİSTİK (enerji, ZCR, centroid, rolloff)

| Öznitelik               | Nedir                                             | Duyguyla ilişkisi                    |
|-------------------------|---------------------------------------------------|--------------------------------------|
| Log RMS enerji          | Aralıktaki ortalama ses şiddeti                   | Kızgın/mutlu yüksek; üzgün düşük     |
| Enerji std              | Şiddetin aralık içi dalgalanması                  | Heyecanlı/patlamalı konuşmada yüksek |
| ZCR (sıfır geçiş oranı) | Dalga'nın saniyede kaç kez eksi-artı değiştirdiği | Sert/tıslamalı seslerde yüksek       |
| Spektral centroid       | Enerjinin "ağırlık merkezi" frekansı              | Parlak/tiz konuşmada yüksek          |
| Spektral rolloff        | Enerjinin %85'inin altında kaldığı frekans        | Spektrumun yukarı yayılımı           |

13+13+13+5 = **44 öznitelik/aralık**. Kod: `final/features.py` → `_interval_features`.



## ARALIK DÜZENİ (Yöntem 2'nin hiperparametreleri)

### NEDİR?

Kayıt N eşit yerleşimli pencereye bölüp her pencereden 44'lük vektör çıkarmak. **N (sayı) ve pencere genişliği (ms) ödevin şart koştuğu iki hiperparametre.**

### NASIL ÇALIŞIR?

Pencere başlangıçları kayda eşit dağıtılır. Kayıt kısa ise pencereler ÖRTÜŞÜR (kayan pencere gibi), uzunsa aralarında boşluk kalır — her iki durumda da tam N satır çıkar, RNN hep aynı boyutta seri görür. Bizim kayıtlar ~2,5 sn; kazanan  $32 \times 200$  ms = 6,4 sn kapsam → pencereler örtüşüyor.

### PROJEDE

Arama 16/24/32 aralık × 200/300/400 ms denedi; **32×200 kazandı** (iyileştirme turunda bulundu) — RNN'i birinci yapan bulgu. "Neden kazandı?" → daha çok ve daha ince pencere, duygunun zaman içindeki değişimini daha yüksek çözünürlükle yakaladı.



## BÖLÜM 3 — Yöntem 1: CNN'i Sıfırdan Anlamak

### ✦ KONVOLÜSYON (evrişim)

#### NEDİR?

Küçük bir sayı tablosunun (filtre, bizde  $3 \times 3$ ) resmin üzerinde adım adım gezmesi; her konumda filtreye o bölgenin sayıları çarpılıp toplanır  $\rightarrow$  tek sayı. Sonuç: "bu desen resmin neresinde var?" haritası.

*Elinde şeffaf bir şablonla fotoğrafın üzerinde gezmek: şablon o bölgeye uyuyorsa alet yüksek ses çıkarır. Şablonun deseni (filtrenin sayıları) eğitimle ÖĞRENİLİR — biz çizmeyiz.*

**Elle mini örnek:** Girdi  $3 \times 3 = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{bmatrix}$ , filtre  $2 \times 2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ .

Sol-üst konum:  $1 \cdot 1 + 2 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 2$ . Sağ-üst:  $2 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 3 \cdot 1 = 5$ . Sol-alt:  $0 + 0 + 0 + 1 = 1$ . Sağ-alt:  $1 + 0 + 0 + 1 = 2$ .  $\rightarrow$  Çıktı  $2 \times 2 = \begin{bmatrix} 2 & 5 \\ 1 & 2 \end{bmatrix}$ . Hepsi bu — CNN'in çekirdek işlemi bu kadar.

#### KANAL

Bir katmanda birden çok filtre çalışır; her filtre kendi haritasını üretir = **kanal**. Bizde  $32 \rightarrow 64 \rightarrow 128$ : ağ derinleştikçe daha çok ve daha soyut desen dedektörü.

### ✦ ReLU, MaxPool, BatchNorm

#### ReLU

$\max(0, x)$ : negatifleri sıfırla, pozitifleri bırak. Ağa "doğrusal olmayanlık" katar — bu olmasa 100 katman bile tek toplama-çarpmaya eşdeğer kalırdı, karmaşık desen öğrenilemezdi.

#### MaxPool( $2 \times 2$ )

Her  $2 \times 2$  bölgeden EN BÜYÜK değeri al  $\rightarrow$  harita yarıya iner. İki faydası: hesap küçülür; "desen tam olarak hangi pikselde" bilgisini gevşetip küçük kaymalara dayanıklılık kazandırır.

*Fotoğrafı uzaktan bakarak özetlemek: detay kaybolur ama "sağ üstte parlak bir şey var" bilgisi kalır — duygu için yeterli olan da bu.*

#### BatchNorm

Her kanalın çıktısını yığın içinde ortalama 0 / yayılım 1'e çeker. Eğitimi kararlı ve hızlı yapar; katmanlar birbirinin sürekli değişen ölçeğine uyum sağlamak zorunda kalmaz. (Testte, eğitim boyunca biriktirdiği ortalamaları kullanır — `model.eval()` bunun için.)

## ✦ GAP (Global Average Pooling) ve BİZİM CNN'İN TAM AKIŞI

### GAP NEDİR?

Son katmandaki her kanal haritasının TEK ortalamasını almak: 128 kanal → 128 sayı. Faydası: kayıt uzunluğu değişse de çıktı boyutu sabit kalır ve parametre patlaması yaşanmaz.

### KATMAN KATMAN BOYUT TAKİBİ (ezber değil, MANTIK)

| Aşama                     | Çıktı boyutu             | Ne oldu                            |
|---------------------------|--------------------------|------------------------------------|
| Girdi                     | $1 \times 64 \times 128$ | tek kanallı log-mel "resmi"        |
| Blok 1 (32 kanal + pool)  | $32 \times 32 \times 64$ | 32 desen haritası, boyut yarılandı |
| Blok 2 (64 kanal + pool)  | $64 \times 16 \times 32$ | daha soyut desenler                |
| Blok 3 (128 kanal + pool) | $128 \times 8 \times 16$ | en soyut desenler                  |
| GAP                       | 128                      | her kanalın ortalaması             |
| Dropout(0,3) + Linear     | 6                        | 6 duygu puanı (logit)              |

Toplam ~287 bin parametre — bilerek küçük: 5.234 örnekle dev ağı beslenmez, ezberler.

## BÖLÜM 4 — Yöntem 2: RNN / GRU'yu Sıfırdan Anlamak

### ✦ RNN — NEDEN "SIRALI" MODEL?

#### NEDİR?

Girdiyi tek seferde değil **adım adım** okuyan ve adımlar arasında **hafıza** (gizli durum) taşıyan ağı. Her adımda: yeni girdiyi al + önceki hafızayla birleştir + hafızayı güncelle.

*Kitap okurken not defteri tutmak: her cümleyi okurken defterine bakar, gerekirse deftere ekleme/silme yaparsın. Son sayfaya geldiğinde defterin kitabın özetidir. Defter = gizli durum.*

#### NEDEN BİZE UYGUN?

Duygu zamanla GELİŞİR: öfke cümlelerin sonunda patlayabilir, üzüntü baştan sona sönük seyredebilir. 32 aralıklık serimiz tam bir "hikâye"; RNN bu hikâyeyi kronolojik okur.

## GRU — KAPILI HAFIZA

### SORUN

Sade RNN uzun serilerde unuttukandır: hafıza her adımda ezilir, baştaki bilgi sona ulaşamaz (gradyan sönmesi). Çözüm: hafızayı **kapılarla** korumak.

### GRU'NUN 2 KAPISI

Güncelleme kapısı z: "defterin ne kadarını KORUYAYIM, ne kadarını yeni bilgiyle değiştireyim?"

Sıfırlama kapısı r: "yeni notu yazarken eski defterin ne kadarına bakayım?"

$new\_defter = (1-z) \cdot eski\_defter + z \cdot yeni\_adad\_not$

Kapılar 0-1 arası değer üretir (sigmoid) ve DEĞERLERİ EĞİTİMLE ÖĞRENİLİR — yani ağ, "neyi hatırlamak önemli" kararını kendisi öğrenir.

### LSTM FARKI

LSTM aynı fikrin 3 kapılı + ayrı hücre-durumlu, daha kalabalık versiyonu. Aramada ikisini de denedik; GRU geçerlemede öndeydi — küçük/orta veride daha az parametreyle benzer güç tipiktir.

## ÇİFT YÖNLÜLÜK (Bi-) ve ZAMAN HAVUZLAMA

### Bi-GRU

Seri hem baştan-sona hem sondan-başa iki ayrı GRU ile okunur; iki "defter" yan yana eklenir ( $192+192=384$  sayı/adım). Bir anın anlamı hem öncesine hem sonrasına bağlı olduğundan kazandırır.

### HAVUZLAMA: 32 adımı tek vektöre nasıl indiririz?

| Tür    | Mantığı                                              | Bizde                          |
|--------|------------------------------------------------------|--------------------------------|
| last   | sadece son adımın defteri                            | baştaki bilgi silikleşebilir   |
| mean ★ | tüm adımların ortalaması                             | KAZANAN — duygu kayda yayılmış |
| max    | her boyutta en güçlü an                              | tek zirveye odaklı             |
| attn   | öğrenilmiş ağırlıklı ortalama ("önemli ana çok bak") | geliştirmede denendi, geçemedi |

## BİZİM BiGRU'NUN TAM AKIŞI + 942.342 PARAMETRENİN HESABI

girdi: 32 aralık  $\times$  44 öznitelik  $\rightarrow$  BiGRU katman 1: her yön 192 birim  $\rightarrow$  çıktı 32  $\times$  384  $\rightarrow$  BiGRU katman 2 (girdisi 384)  $\rightarrow$  çıktı 32  $\times$  384  $\rightarrow$  mean pooling (32 adımın ortalaması)  $\rightarrow$  384  $\rightarrow$  Dropout(0,3) + Linear(384 $\rightarrow$ 6)  $\rightarrow$  6 duygu puanı

### Parametre sayısı nereden geliyor? (hoca sorabilir)

GRU'da kapı+adad için 3 ağırlık takımı var. Katman 1, yön başına:  $3 \cdot 192 \cdot 44$  (girdi ağırlıkları) +  $3 \cdot 192 \cdot 192$  (hafıza ağırlıkları) +  $2 \cdot 3 \cdot 192$  (bias) = 137.088; iki yön  $\rightarrow$  274.176. Katman 2 (girdisi 384):  $3 \cdot 192 \cdot 384$  +  $3 \cdot 192 \cdot 192$  + 1.152 = 332.928; iki yön  $\rightarrow$  665.856. Kafa:  $384 \cdot 6 + 6 = 2.310$ . **Toplam = 942.342** — winner.json'daki sayının birebir kendisi.

## BÖLÜM 5 — Eğitim Mutfağı (formüller + gerçek sayılarla)

### ★ SOFTMAX

#### NEDİR?

Modelin 6 ham puanını (logit), toplamı 1 olan olasılıklara çevirir:  $p_i = e^{z_i} / \sum e^{z_j}$ .

**Sayısal örnek:** logits = [2,0 ; 1,0 ; 0,1] → e'ler = [7,39 ; 2,72 ; 1,11], toplam 11,21 → olasılıklar = [0,66 ; 0,24 ; 0,10]. En büyük logit en büyük olasılık olur ama fark "yumuşak" paylaşılır — adı buradan.

### ★ SINIF-AĞIRLIKLILIK CROSS-ENTROPY (yönergenin açık şartı)

#### SORUN

Nötr sınıfın kaydı daha az (1.087'ye karşı diğerleri 1.271). Ağırlıksız loss'ta model "çoğunluğa oynayıp" azınlığı ihmal edebilir.

#### ÇÖZÜM

$$w_k = n / (K \cdot n_k) \rightarrow L = -w_{\text{doğru}} \cdot \log(p_{\text{doğru}})$$

**Bizim GERÇEK sayılarla:**  $n=7.442$ ,  $K=6$ .

$$w_{\text{nötr}} = 7442 / (6 \times 1087) = \mathbf{1,141} \quad \bullet \quad w_{\text{kızgın}} = 7442 / (6 \times 1271) = \mathbf{0,976}.$$

Yani nötr bir örneği yanlış yapmak, kızgın bir örneği yanlış yapmaktan ~%17 daha pahalı → model azınlığı ihmal edemez. Kod: `final/training.py` → `inverse_frequency_weights`.

### ★ DROPOUT

#### NEDİR?

Eğitim sırasında nöronların rastgele bir kısmını (bizde %30) O ADIM İÇİN kapatmak. Her adımda farklı bir "eksik takım" çalışır → hiçbir nöron tek başına vazgeçilmez olamaz, bilgi dağıtık öğrenilir.

*Antrenmanda her gün rastgele 3 oyuncuyu dinlendirmek: takım hiçbir yıldıza bağımlı kalmaz. MAÇTA (testte) herkes sahada — `model.eval()` dropout'u kapatır.*

### ★ WEIGHT DECAY, GRADYAN KIRPMA, AMP

**Weight decay (1e-4):** her adımda ağırlıkları birazcık sıfıra çekme cezası → aşırı büyük, ezberci ağırlıklar oluşamaz. **Gradyan kırpma (norm 5):** tek bir bozuk yığından gelen dev gradyanın eğitimi raydan çıkarmasını önleyen emniyet kemeri. **AMP:** GPU'da hesapların bir kısmını 16-bit yapıp hız kazandıran mod; sonucu değiştirmez, süreyi kısaltır.

## 🚀 SPECAUGMENT (geliştirme aşamasının kazananı)

### NEDİR?

Eğitim sırasında spectrogram "resminde" rastgele yatay (frekans) ve dikey (zaman) şeritleri silmek (bizde 2+2 şerit). Model, bilginin bir kısmı yokken bile doğru karar vermeyi öğrenir → ezber kırılır.

*Bazı kelimeleri bantla kapatılmış metinden anlam çıkarmaya alıştırmak: öğrenci bağlamdan tamamlamayı öğrenir; tek kelimeye bel bağlamaz.*

### PROJEDE

Geçerlemede kazananı geçti (+0,0075) → teste alındı: %62,6 → %62,9. Aynı mantıkla RNN'e gürültü ekledik ama geçerlemede KAZANAMADI → teste götürmedik; bu olumsuz sonucu da raporladık (bilimsel dürüstlük). Kural: artırma yalnız EĞİTİM yığınlarına.

## BÖLÜM 6 — Ölçme: Sonuçları Nasıl Okuyoruz?

## 🚀 KARIŞIKLIK MATRİSİ (confusion matrix)

### NASIL OKUNUR?

Satır = GERÇEK sınıf, sütun = TAHMİN. Köşegen (sol üst → sağ alt) = doğrular. Köşegen dışı her hücre "gerçek X'lerin ne kadarı Y sanıldı" demektir. Bizimki satır bazında normalize: her satır toplamı 1.

### BİZİM MATRİSTEN OKUMALAR

Köşegen 0,61-0,67 → altı sınıfta dengeli başarı (ağırlıklı loss işe yaradı). En büyük köşegen-dışı değerler: mutlu→korku 0,22 ve korku→üzgün 0,20 → benzer "uyarılma düzeyi"ndeki duygular akustik olarak birbirine benziyor. En zayıf sınıf: disgust.

## 🚀 PRECISION, RECALL, F1, MACRO-F1, DENGELİ DOĞRULUK

Precision =  $TP/(TP+FP)$ : "kızgın DEDİKLERİMİN kaç gerçekten kızgın?" (yanlış alarm kontrolü)

Recall =  $TP/(TP+FN)$ : "GERÇEK kızgınların kaçını yakaladım?" (kaçırma kontrolü)

F1 =  $2PR/(P+R)$ : ikisinin dengesi (küçük olan cezalandırılır)

Macro-F1 = 6 sınıfın F1 ortalaması • Dengeli doğruluk = 6 sınıfın recall ortalaması

**%90 tuzağı (hocanın klasik sorusu):** 100 örnek: 90 A + 10 B. Model HEPSİNE "A" desin → doğruluk %90 (süper görünür!) ama B recall'u 0 → dengeli doğruluk %50, B'nin F1'i 0. **Ders:** dengesiz veride doğruluk yalan söyler; macro-F1/dengeli doğruluk şart. MELD'de bunu canlı gördük: doğruluk %44 ama dengeli doğruluk %20 → model nötr'e kaçıyor.

## BÖLÜM 7 — Deney Süreci, Sonuçlar ve Önceki Ödevlerle Hikâye

### 7.1 Üç aşamalı süreç

1. **ARAMA:** CNN 11 aday, RNN 16 aday (aralık düzeni dahil) deneme sınavında (geçerleme) yarıştı.
2. **YEREL İYİLEŞTİRME:** kazananın her ayarı bir tık oynatıldı (CNN 8, RNN ~8 aday). RNN'de aralık  $24 \times 300 \rightarrow 32 \times 200$  ms burada bulundu — birinciliği getiren hamle.
3. **GELİŞTİRME:** SpecAugment (CNN ✓) ve dikkat/gürültü (RNN X). Yalnız geçerlemeyi geçen teste gitti.

### 7.2 Sonuç tabloları (ezber)

| Model                    | Val macro-F1  | Test doğruluk | Test macro-F1 |
|--------------------------|---------------|---------------|---------------|
| Yöntem 1: CNN            | 0,5531        | 0,6264        | 0,6282        |
| Yöntem 1 + SpecAugment   | 0,5606        | 0,6292        | 0,6301        |
| <b>Yöntem 2: BiGRU ★</b> | <b>0,5638</b> | <b>0,6377</b> | <b>0,6440</b> |

Bölme: 5.234 / 1.148 / 1.060 (aktöre göre, seed 42). Referanslar: şans ~%17, insan %40,9, sıfırdan tavan %68,1 (5'li ansambl), ön eğitilmiş dev modeller %58-77.

### 7.3 Önceki ödevlerden bugüne (hoca "önceki aşamalarla karşılaştı" diyebilir — yönergede var)

| Aşama        | Yaklaşım                                | Sonuç              | Ders                                                      |
|--------------|-----------------------------------------|--------------------|-----------------------------------------------------------|
| Ödev 1       | wav2vec2 öznitelik (donmuş) + PCA + KNN | alt kümede orta    | klasik başlangıç; öğrenme yok, sadece komşuluk            |
| Ödev 2       | Klasik ML (Gradient Boosting, RF...)    | GB ~%52 (alt küme) | elle özniteliklerle tavana yaklaşıldı                     |
| Ödev 3       | MLP (mel vektörü)                       | %44 (alt küme)     | düz katmanlar uzamsal/zamansal yapıyı kullanamıyor        |
| <b>Final</b> | <b>CNN + BiGRU (tam veri)</b>           | <b>%62,6-63,8</b>  | yapıya uygun mimari + tam veri + doğru protokol = sıçrama |

**Şerh:** önceki ödevler ~2,6 binlik ALT KÜMEDE koştı; sayılar doğrudan değil YÖN olarak kıyaslanır (bunu sen söyle, dürüstlük puanı).

### 7.4 MELD genellemesi

Aynı kod, sıfır değişiklikle, dizi diyaloglarından oluşan MELD'de: CNN %44,3 / BiGRU %38,4; dengeli doğruluk ~%20 → model nötr'e kaçıyor. Neden düşük: arka plan gürültüsü/gülüşme, çok kısa kayıtlar, doğal konuşma, nötr baskınlığı. Çıkarım: **sınır yöntemde değil veri koşullarında**; literatür bu yüzden MELD'e metin/görüntü kipleri ekler.

## BÖLÜM 8 — Ders Konularıyla Eşleşme (finalin dersle bağı)

| Ders slaytı              | Projede karşılığı                                                    | Kitapçıkta |
|--------------------------|----------------------------------------------------------------------|------------|
| 03 — Data and Features   | Mel/MFCC çıkarımı, z-skor, bölme, sızıntı önlemleri                  | Bölüm 1-2  |
| 04 — Evaluation Metrics  | Karışıklık matrisi, precision/recall/F1, macro-F1, dengeli doğruluk  | Bölüm 6    |
| 05 — Supervised Learning | Etiketli veriyle sınıflandırma kurulumunun tamamı                    | Bölüm 0-1  |
| 10 — Neural Networks     | Katman, aktivasyon, loss, geri yayılım, optimizasyon                 | Bölüm 1, 5 |
| 11 — CNN                 | Konvolüsyon, pooling, BatchNorm, GAP — Yöntem 1'in tamamı            | Bölüm 3    |
| 12 — RNN                 | Gizli durum, GRU/LSTM kapıları, çift yönlülük — Yöntem 2'nin tamamı  | Bölüm 4    |
| 07 — Ensemble            | "Nasıl daha iyi olur?" cevabında: Ristea'nın %68'i 5 modelli ansambl | Bölüm 10   |

💡 Bu tablo final SINAVI çalışırken de işine yarar: proje, dersin 5 büyük konusunun uygulamalı halidir.

## BÖLÜM 9 — Sunum Metni (slayt slayt, ~5 dk)

**Kural:** link/kaynak slaytlarında durma; asıl vakti ★ slaytlarda harca. Okuma — konuş.

### 1. Kapak

5 sn

"Merhaba, ben Ahmet. Projem konuşmadan duygu tanıma — CREMA-D üzerinde iki yöntemi karşılaştırdım."

### 2. Bağlantılar

5 sn

"Veri seti, kod ve tüm çalışma bu bağlantılarda paylaşıldı."

### 3. Ajanda

5 sn

"Sırayla: problem, iki yöntem, deney süreci, sonuçlar."

### 4. Problem

20 sn

"Amaç, kısa ses kaydından duyguyu bulmak — 6 sınıf. Zor çünkü aynı cümle altı duyguyla söylenebiliyor; içerik değil söyleyiş önemli. Kural gereği hazır model yok, her şey sıfırdan."

### ★ 5. Veri + Protokol

30 sn

"7.442 kayıt, 91 aktör. İnsanlar bile yalnız sestten %41'de. En kritik kararım: veriyi aktöre göre böldüm — aynı ses iki tarafta olursa model duyguyu değil sesi ezberler. %90'lık internet sonuçları bu hatadan." (kırmızı kutuyu göster)

### 6. Yöntem 1

25 sn

(şemayı elle takip) "Sesi log-mel spectrogram resmine çevirip sıfırdan yazdığım küçük bir CNN ile sınıflandırıyorum. Kazanan: 32-64-128 kanal."

## ★ 7. Yöntem 2

30 sn

"Sesi eşit aralıklara bölüyorum — aralık sayısı ve genişliği HİPERPARAMETRE, ödevin şartı. Aralık başına 44 öznitelik, sonra çift yönlü GRU. Yeşil kutu: bu iki hiperparametrenin aranması birinciliği getirdi."

## 8. Eğitim

20 sn

"Ağırlıklı loss + erken durdurma; normalizasyon yalnız eğitimden — sızıntı yok. 43 aday eğittim; test'e sadece kazananlar dokundu."

## ★ 9. SONUÇLAR

40 sn

"CNN %62,6; SpecAugment ile %62,9; kazanan BiGRU %63,8. Bağlam: insan %41, sıfırdan tavan %68 — 4 puan mesafedeyiz."

## 10. Eğriler

15 sn

"Eğitim kaybı düşerken geçişleme dönüyor — erken durdurma modeli tam dönüş noktasında seçti."

## 11. Matrisler

15 sn

"Köşegen 0,61-0,67, dengeli. En çok karışanlar: mutlu→korku, korku→üzgün."

## ★ 12. Geliştirme

25 sn

"SpecAugment geçerlemede kazandı → teste gitti. RNN varyantları kazanamadı → teste GÖTÜRMEDİM; olumsuz sonucu da raporladım."

## 13. MELD

15 sn

"Aynı kod MELD'de: %44 ve %38. Düşük ama beklenen — sınır yöntemde değil veride."

## 14. Literatür çizgisi

20 sn

(yeşil nokta) "İnsanın 22 puan üstü, sıfırdan tavana 4 puan mesafe."

## 15. Sonuç → 16-17. Kaynaklar → 18. Teşekkür + DEMO

"İki yöntem, aynı adil protokol, tekrar üretilebilir sonuçlar... Teşekkürler — canlı da gösterebilirim." → hazır terminalde tek Enter.

# BÖLÜM 10 — Soru Bankası (cevaplarıyla)

## Protokol

### Neden aktöre göre bölme?

"Aynı ses iki tarafta olursa model duyguyu değil sesi ezberler; skor 20-30 puan şişer."

### Neden macro-F1 ile seçim?

"Sınıflar dengesiz; doğrulukla seçim modeli çoğunluğa kaydırırdı. Macro-F1 her sınıfa eşit söz hakkı verir."

### Val 0,55 iken test neden 0,63?

"Konuşmacı-bazlı bölmede test konuşmacıları şans eseri biraz kolay çıkmış — normal varyans. Tüm seçimler yine geçerlemeyle yapıldı."



## Tekrar üretilebilir mi?

"Evet: sabit seed 42, deterministik bölme, kayıtlı modeller. Notebook testi yeniden koşuyor, kayıtlı metriklerle birebir aynı."

## Ses / öznelik

### Neden 16 kHz?

"Nyquist gereği 8 kHz'e kadar frekans temsil edilir; konuşmadaki duygu bilgisi bu bantta. Daha yükseği gereksiz yük."

### Mel neden var?

"Kulak frekansı logaritmik algılar; mel ölçeği bu algıyı taklit eder — alçak frekansa sık, yükseğe seyrek filtre."

### MFCC nedir, neden 13?

"Log-mel spektrumunun DCT ile sıkıştırılmış hali; ilk 13 katsayı tınının kaba şeklini taşır, üst katsayılar detay/gürültüdür — 13 konuşma işlemede standarttır."

### 44 öznelik neler?

"13 MFCC ort + 13 MFCC std + 13 delta ort + log enerji, enerji std, ZCR, centroid, rolloff."

### Aralıklar örtüşüyor mu?

"Kayıt ~2,5 sn, 32×200 ms = 6,4 sn kapsam → evet, kayan pencere gibi örtüşüyor; uzun kayıta boşluk kalırdı. Her durumda 32 satır sabit."

## Mimari

### CNN'ini anlat.

"3 blok × (2 adet 3×3 Conv-BN-ReLU + MaxPool), kanallar 32-64-128, sonra GAP + dropout + linear. ~287 bin parametre, sıfırdan."

### GAP neden?

"Kayıt uzunluğu değişse de sabit boyutlu çıktı verir ve flatten+dense'e göre çok az parametre → ezber riski düşer."

### GRU-LSTM farkı?

"GRU 2 kapı, LSTM 3 kapı + ayrı hücre durumu. İkisini de aradım; GRU geçerlemede önde çıktı — az parametreyle benzer güç."

### Neden çift yönlü?

"Bir anın anlamı hem öncesine hem sonrasına bağlı; iki yönün defterleri birleşince bağlam tamamlanıyor."

### Mean pooling neden kazandı?

"Duygu kaydın geneline yayılıyor; ortalama tüm serinin kanıtını topluyor. Attention denedim, geçerlemede geçemedi."

### RNN neden CNN'i geçti?

"Aralık düzeni hiperparametre olarak arandı ve 32×200 bulundu — veriye uygun zaman çözünürlüğü. Fark küçük (1,1 puan); asıl belirleyici protokol."

## Eğitim

### Early stopping mekaniğinin?

"Her epoch geçerleme macro-F1; en iyiyse ağırlık kopyası; 8 epoch iyileşme yoksa dur, en iyi anı geri yükle. CNN 37 koştu, 29'u aldık."

### Ağırlıklı loss formülü?

" $w = n / (K \cdot n_k)$ : nötr 1,141, diğerleri 0,976 — nötr hatası ~%17 daha pahalı."

### AdamW nedir, Adam'dan farkı?

"Uyarlanır adım boylu gradyan inişi; W'si weight decay'in doğru (gradyandan ayrık) uygulanması."

### BatchNorm testte nasıl çalışır?

"Eğitim boyunca biriktirdiği yürüyen ortalama/varyansı kullanır; model.eval() bu moda geçirir."

### Freeze yaptın mı?

"O not aktarmalı öğrenme için; ses kategorisinde aktarmalı öğrenme yasak — dondurulacak hazır katman yok. Erken durdurma doğrudan uygulandı."

### Hazır model var mı?

"Hayır. librosa yalnız öznitelik çıkarımı (yönerge izinli); modeller torch.nn'den sıfırdan."

## Analiz

### En zayıf sınıf?

"Disgust — belirgin akustik imzası yok; sönük hali nötr/üzgüne, sert hali kızgına benziyor. Demo kötü örneklerimin 3'ü disgust."

### MELD neden düşük?

"Gürültü/gülüşme, çok kısa kayıtlar, doğal konuşma, nötr baskınlığı. Doğruluk %44 ama dengeli doğruluk %20 → nötr'e kaçış. Ses-tek-kipli MELD literatürde de düşüktür."

### Nasıl daha iyi olurdu?

"Kurallar izin verseydi: önceden eğitilmiş wav2vec2/HuBERT (+8-12 puan). Kurallar içinde: model ansamblı (%68 öyle alındı), daha geniş artırma, pencere-oylama."

### Bilmediğin soru?

Uydurma → "Şu an net cevabını veremem ama şöyle test ederdim: ..."

## BÖLÜM 11 — Mini Quiz (kendini sına — cevaplar en altta)

1. Eğitim/geçerleme/test üçlüsünün her birinin görevi nedir?
2. Konuşmacı-bağımsız bölme olmasaydı skorumuz neden şişerdi?
3. Early stopping'de "patience 8" ne demek?
4.  $w_k = n / (K \cdot n_k)$  formülünde nötr'ün ağırlığı neden 1'den büyük çıkar?
5. MaxPool ile GAP arasındaki fark nedir?
6. GRU'nun iki kapısının isimleri ve görevleri?
7.  $32 \times 200$  ms neyi ifade eder ve neden önemliydi?
8. Doğruluk %90 iken dengeli doğruluk %50 olabilir mi? Nasıl?
9. SpecAugment neden yalnız eğitim yığınlarına uygulanır?
10. z-skor parametreleri neden yalnız eğitim kümesinden hesaplanır?

**Cevap anahtarı:** 1) öğren / ayar seç / son ölçüm. 2) Model duyguyu değil tanıdık sesi ezberlerdi. 3) Üst üste 8 epoch iyileşme yoksa dur. 4) Nötr'ün örneği az (1.087) → payda küçük → ağırlık büyük. 5) MaxPool bölgesel en büyüğü alıp haritayı küçültür; GAP tüm haritanın tek ortalamasını alır. 6) Güncelleme (hafızanın ne kadarı korunacak) ve sıfırlama (yeni not yazarken eskiye ne kadar bakılacak). 7) Yöntem 2'nin kazanan aralık düzeni: 32 pencere × 200 ms — iyileştirme turunda bulundu ve birinciliği getirdi. 8) Evet: dengesiz veride hep çoğunluğu tahmin eden model (mini örnekteki gibi). 9) Geçerleme/test bozulursa ölçüm güvenilirmez olur; artırma sadece öğrenmeyi zorlaştırmalı, ölçmeyi değil. 10) Test istatistiği kullanmak test bilgisini eğitime sızdırmak olur (leakage).

## BÖLÜM 12 — Sözlük (son gece hızlı tekrar)

| Terim                            | Tek cümle                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------|
| Model / parametre                | Ayarlanabilir sayılarla dolu tahmin makinesi / o sayıların kendisi.                        |
| Epoch / batch                    | Verinin tam turu / verinin 32-64'lük işlem grupları.                                       |
| Loss (cross-entropy)             | $-\log(p_{\text{doğru}})$ ; "emin ve yanlış" en pahalı.                                    |
| Gradyan / öğrenme oranı          | Hatayı azaltan yön pusulası / o yöndeki adım boyu.                                         |
| AdamW                            | Uyarlanır adımlı optimizier + doğru weight decay.                                          |
| Hiperparametre                   | Eğitimle öğrenilmeyen, bizim seçtiğimiz ayar.                                              |
| Overfitting                      | Eğitimi ezberleyip yeni veride çuvallamak.                                                 |
| Early stopping                   | Deneme sınavı düzelmeyince durup en iyi ana dönmek.                                        |
| Leakage                          | Test bilgisinin karar sürecine sızması.                                                    |
| Z-skor                           | $(x - \text{ort}) / \text{std}$ ; herkesi aynı cetvele çekme (yalnız eğitimden öğrenilir). |
| Spectrogram / mel / MFCC         | Zaman-frekans haritası / kulak cetveli / 13 sayılık tını parmak izi.                       |
| Konvolüsyon / kanal              | Kayan desen dedektörü / dedektör başına çıkan harita.                                      |
| ReLU / MaxPool / BatchNorm / GAP | Doğrusal olmayanlık / bölgesel özet / kararlılaştırıcı / kanal ortalaması.                 |
| GRU / BiGRU                      | 2 kapılı hafızalı hücre / seriyi iki yönden okuyan hali.                                   |
| Pooling (zaman)                  | 32 adımı tek vektöre indirme (last/mean/max/attn).                                         |
| Softmax                          | Puanları toplamı 1 olan olasılıklara çevirme.                                              |
| Precision / Recall / F1          | Dediklerimin doğruluğu / gerçekleri yakalama / ikisinin dengesi.                           |
| Macro-F1 / dengeli doğruluk      | Sınıf F1 ortalaması / sınıf recall ortalaması.                                             |
| SpecAugment                      | Eğitimde spectrogram şeridi silme artırması.                                               |
| Seed                             | Rastgeleliği sabitleyen tohum → tekrarlanabilirlik.                                        |

## BÖLÜM 13 — Çalışma Planı ve Sunum Günü Listesi

### Önerilen çalışma planı (3 oturum):

1. **Oturum 1 (kavramlar):** Bölüm 0-1-2'yi oku; her kavramda "benzetmeyi kendi cümlelele bir kez söyle". Ardından VS Code'da `final/features.py` 'yi aç, yorumları okuyarak Bölüm 2'yi kodda gör.
2. **Oturum 2 (yöntemler + ölçme):** Bölüm 3-4-5-6; yanında `models.py` ve `training.py` . Mini quiz'i çöz — yanlışlarının bölümüne geri dön.
3. **Oturum 3 (sunum):** Bölüm 7-8-9'u oku; sunumu 2-3 kez SESLİ prova et (süre tut); Bölüm 10'daki soruları kendine sorup SESLİ cevapla.

### Sunum günü:

- ☐ Terminal açık, komut yazılı, Enter'a basılmamış:  

```
cd "C:\Users\user\Desktop\470 proje\speech-emotion-recognition"  
.\.venv\Scripts\python.exe -X utf8 final\demo.py --rastgele 3
```
- ☐ Kötü örnek komutu yedekte: `...demo.py final\demo_ornekleri\1013_TIE_DIS_XX.wav`
- ☐ Notebook bir kez açılıp kernel seçilmiş: `final\SONUCLAR_VE_DEMO.ipynb`
- ☐ Drive'da final dosyaları yüklü (slayt 2 linki!) • ☐ Şarj + USB yedek • ☐ Su, yavaş konuş, elinle göster.

**Üç altın kural:** (1) Abartma — "sıfırdan modeller için üst bant" de. (2) Zayıflığı önce SEN söyle (disgust, MELD). (3) Bilmediğini uydurma — "şöyle test ederdim" kalıbı. Her iddianın kanıtı kodda/raporda: "final/training.py'de gösterebilirim" demek en büyük kozun.